JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class MemoryHandler

java.lang.Object
    java.util.logging.Handler
        java.util.logging.MemoryHandler

```
public class MemoryHandler
extends Handler
```

`Handler` that buffers requests in a circular buffer in memory.

Normally this `Handler` simply stores incoming `LogRecords` into its memory buffer and discards earlier records. This buffering is very cheap and avoids formatting costs. On certain trigger conditions, the `MemoryHandler` will push out its current buffer contents to a target `Handler`, which will typically publish them to the outside world.

There are three main models for triggering a push of the buffer:

- An incoming `LogRecord` has a type that is greater than a pre-defined level, the `pushLevel`.
- An external class calls the `push` method explicitly.
- A subclass overrides the `log` method and scans each incoming `LogRecord` and calls `push` if a record matches some desired criteria.

Configuration: By default each `MemoryHandler` is initialized using the following `LogManager` configuration properties where `<handler-name>` refers to the fully-qualified class name of the handler. If properties are not defined (or have invalid values) then the specified default values are used. If no default value is defined then a RuntimeException is thrown.

- <handler-name>.level specifies the level for the `Handler` (defaults to `Level.ALL`).
- <handler-name>.filter specifies the name of a `Filter` class to use (defaults to no `Filter`).
- <handler-name>.size defines the buffer size (defaults to 1000).
- <handler-name>.push defines the `pushLevel` (defaults to `level.SEVERE`).
- <handler-name>.target specifies the name of the target `Handler` class. (no default).

For example, the properties for `MemoryHandler` would be:

- java.util.logging.MemoryHandler.level=INFO
- java.util.logging.MemoryHandler.formatter=java.util.logging.SimpleFormatter

For a custom handler, e.g. com.foo.MyHandler, the properties would be:

- com.foo.MyHandler.level=INFO
- com.foo.MyHandler.formatter=java.util.logging.SimpleFormatter

Since:

1.4

Constructor Summary

Constructors

Constructor	Description
`MemoryHandler()`	Create a `MemoryHandler` and configure it based on `LogManager` configuration properties.
`MemoryHandler(Handler target, int size, Level pushLevel)`	Create a `MemoryHandler`.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`void`	`close()`	Close the `Handler` and free all associated resources.
`void`	`flush()`	Causes a flush on the target `Handler`.
`Level`	`getPushLevel()`	Get the `pushLevel`.
`boolean`	`isLoggable(LogRecord record)`	Check if this `Handler` would actually log a given `LogRecord` into its internal buffer.
`void`	`publish(LogRecord record)`	Store a `LogRecord` in an internal buffer.
`void`	`push()`	Push any buffered output to the target `Handler`.
`void`	`setPushLevel(Level newLevel)`	Set the `pushLevel`.

Methods declared in class Handler

getEncoding, getErrorManager, getFilter, getFormatter, getLevel, reportError, setEncoding, setErrorManager, setFilter, setFormatter, setLevel

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

MemoryHandler

```
public MemoryHandler()
```

Create a `MemoryHandler` and configure it based on `LogManager` configuration properties.

MemoryHandler

```
public MemoryHandler(Handler target,
                     int size,
                     Level pushLevel)
```

Create a `MemoryHandler`.

The `MemoryHandler` is configured based on `LogManager` properties (or their default values) except that the given `pushLevel` argument and buffer size argument are used.

Parameters:

`target` - the Handler to which to publish output.

`size` - the number of log records to buffer (must be greater than zero)

`pushLevel` - message level to push on

Throws:

`IllegalArgumentException` - if `size is <= 0`

Method Details

publish

```
public void publish(LogRecord record)
```

Store a `LogRecord` in an internal buffer.

If there is a `Filter`, its `isLoggable` method is called to check if the given log record is loggable. If not we return. Otherwise the given record is copied into an internal circular buffer. Then the record's level property is compared with the `pushLevel`. If the given level is greater than or equal to the `pushLevel` then `push` is called to write all buffered records to the target output `Handler`.

Specified by:

`publish` in class `Handler`

Parameters:

`record` - description of the log event. A null record is silently ignored and is not published

push

```
public void push()
```

Push any buffered output to the target `Handler`.

The buffer is then cleared.

flush

```
public void flush()
```

Causes a flush on the target `Handler`.

Note that the current contents of the `MemoryHandler` buffer are **not** written out. That requires a "push".

Specified by:

`flush` in class `Handler`

close

```
public void close()
```

Close the `Handler` and free all associated resources. This will also close the target `Handler`.

Specified by:

`close` in class `Handler`

setPushLevel

```
public void setPushLevel(Level newLevel)
```

Set the pushLevel. After a LogRecord is copied into our internal buffer, if its level is greater than or equal to the pushLevel, then push will be called.

Parameters:

newLevel - the new value of the pushLevel

getPushLevel

```
public Level getPushLevel()
```

Get the pushLevel.

Returns:

the value of the pushLevel

isLoggable

```
public boolean isLoggable(LogRecord record)
```

Check if this Handler would actually log a given LogRecord into its internal buffer.

This method checks if the LogRecord has an appropriate level and whether it satisfies any Filter. However it does **not** check whether the LogRecord would result in a "push" of the buffer contents. It will return false if the LogRecord is null.

Overrides:

isLoggable in class Handler

Parameters:

record - a LogRecord (may be null).

Returns:

true if the LogRecord would be logged.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.